

ICT2113 SOFTWARE REQRS: NOW & NEAR FUTURE

11. UseCase2.0: Heavyweight RCM Updated To Lightweight

Prof Kevin



“What we need is not evolution but revolution for it to be transformed into something else.”

Bring on the learning revolution!, TED Talk, 2010

Ivar Jacobson never said this, but I believe he would have. Inspired by new mindset that Agile brought to sw lightweight development, he revolutionized his familiar heavyweight Use Case 1987 into entirely new incremental and iterative lightweight RCM. Highlight of UseCase2.0 is it provides new dimension to SDM (like super-Epic of numerous dev cycles) as well as alternative RCM (slices) to User stories. Let's get to it.

Agenda: UseCase2.0 revived for Agile

1. Previous week's learning.
2. Today's learning outcomes.
3. Today's new learning material.
4. Active learning – ques on applying UseCase2.0 + effect.
5. Epilogue.

Previous week's learning

Seven slides

Remember **why** review of previous week's learning **important!**

Strengthen memory retention.

Highlight knowledge gaps.

Prepare for acquiring new material (per **iterative** learning approach in HE institutions).

Timeline of lightweight SDMs evolution

Since 1950, SDM-RCM pairing was heavyweight. In late 1970s, lightweight SDM-RCMs began emerging, though they retained vestiges of heavyweight. Finally, 1996 saw **first truly lightweight** RCM: User stories. Used in C3 project, it went largely unnoticed. Yet, within three years, it would become mainstay of lightweight projs. These are highlights of **evolution of lightweight**.

Year	SDM	RCM
1957	Incremental by Gerald Weinberg & Bernie Dimsdale at IBM	RE
1977	Joint Application Dev by Chuck Morris & Tony Crawford at IBM	RE
1977	V-chart by J.B. Munson at Sys Dev Corp	RE
1986	Spiral Model by Barry Boehm at TRW Software Productivity Sys	Progressive in ea spiral
1991	Rapid Application Development by James Martin	Initial for MVP, progressive after ea iteration
1995	Scrum by Schwaber and Sutherland at Easel Corp	BRUF (not RE)
1996	Extreme Programming (XP) by Kent Beck et al at Chrysler	User story
1997	Feature-Driven by Jeff DeLuca at Singapore's UOB Bank	Initial for PM, progressive with UML models

User story: description not specification

Informal description of new sw's capability/condition from perspectives of its **user**, imagined **usage**, and expected **benefit**, along with its production-grade **code** being **realized** (designed, coded & accepted by stakeholder) within hours to few days.

Conversation. Story initiates expansive **conversation** between user & developer to specify reqr; only source code & some design **saved** after realization.

Epic & Feature. Each sw dev proj has many user stories because each applies to very small part of new sw. To cope with such, stories **amalgamated** into “**Epics**” to manage numerous disparate tasks, or “**Features**” to integrate code blocks into functions. But stories always realized individually.

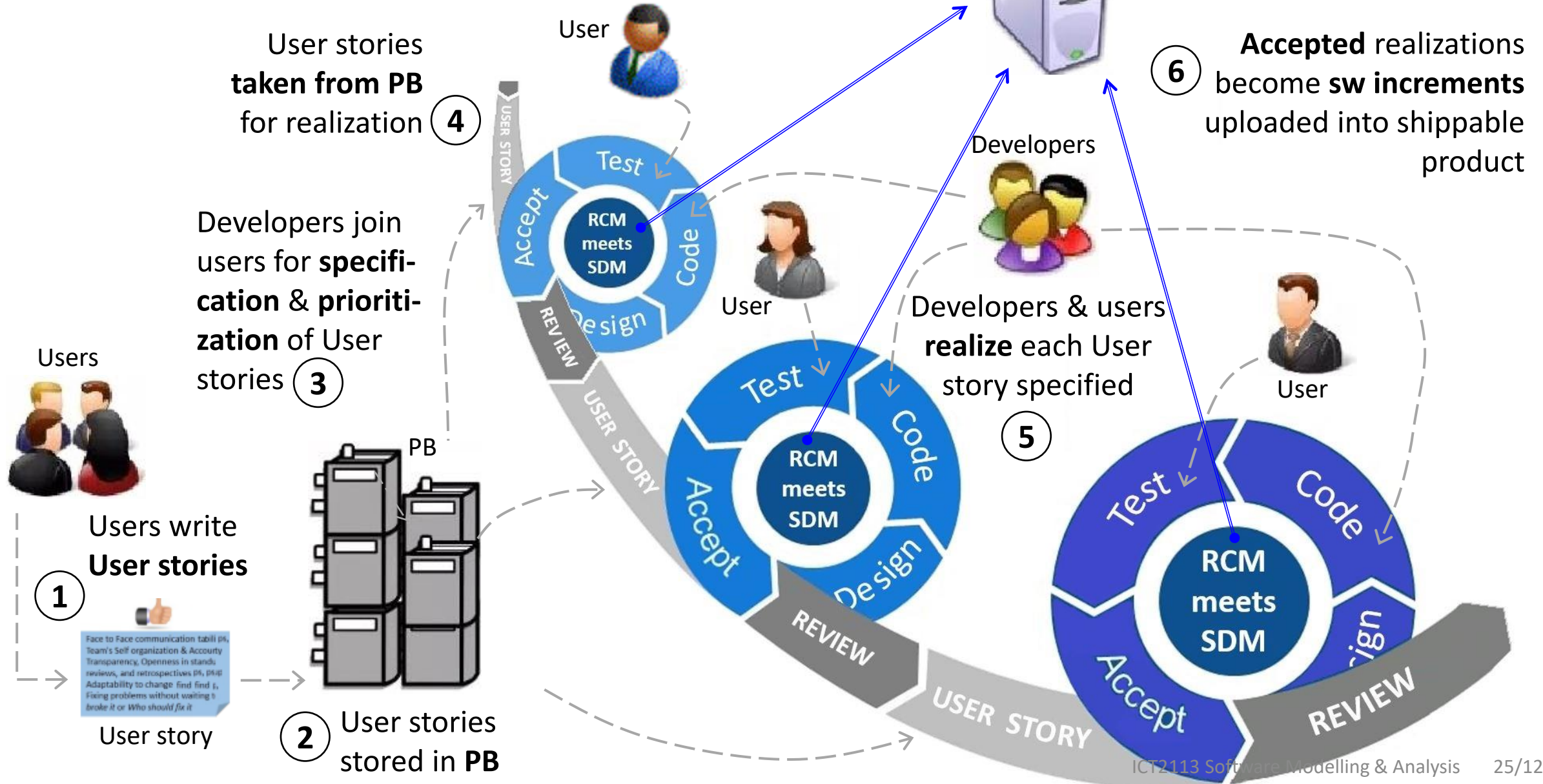
Product backlog. Stories held in repository ‘**Product Backlog**’ (**PB**) for realization.

Not saved. Once chosen for realization, User story deleted from PB.

Coach. First time customers hire ‘**Agile coaches**’ to align corporate expectations with Agile development norms, especially **fragmentation debt**.

Factsheet of fundamental ideas about User story method

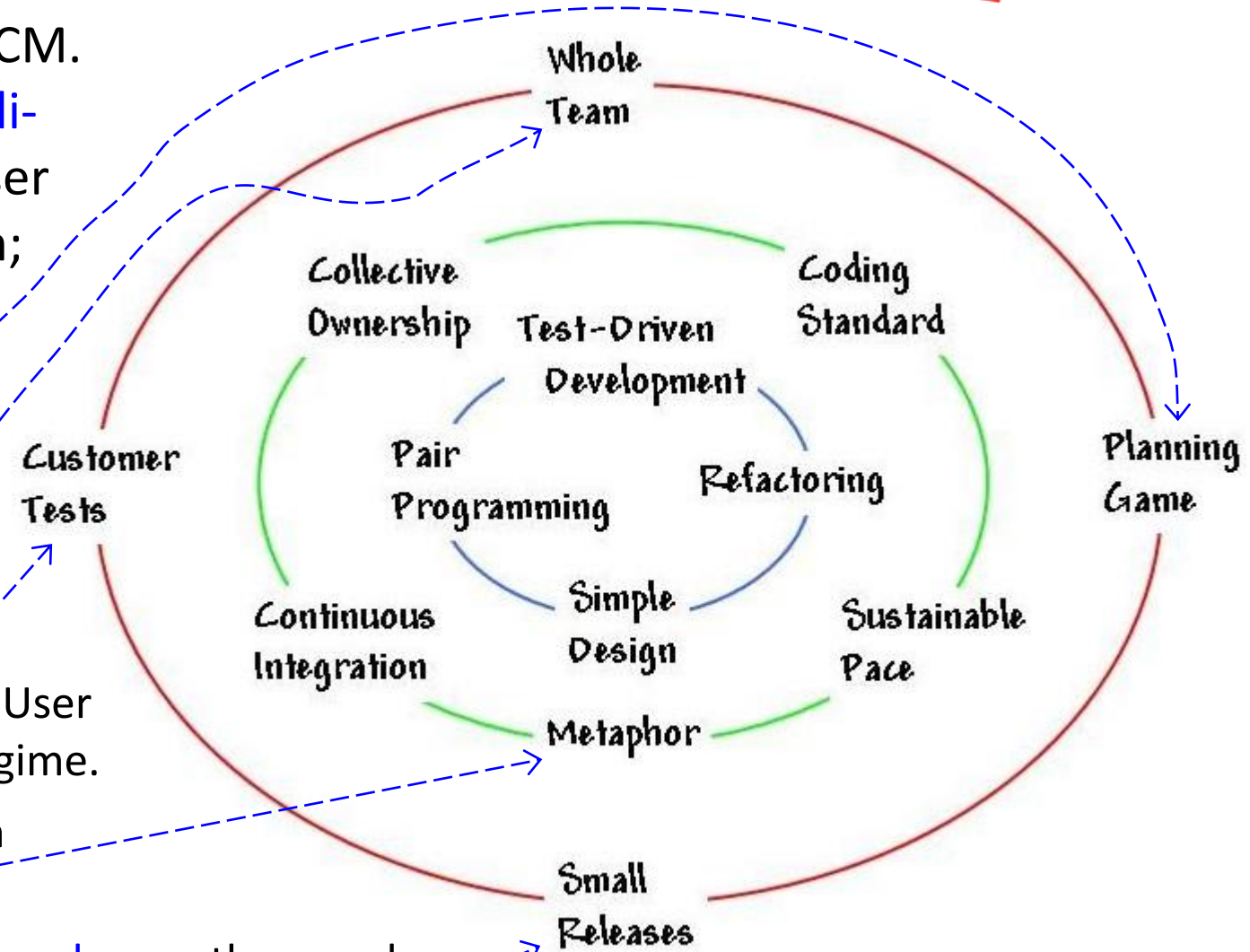
User story process model



User story's origin is embedded in XP practice

User stories are **super-tightly bound** to XP SCM. Of original 13 practices, five (see right) are **directly driving** User stories. For XP teams, User stories are written as they always have been; effectiveness is non-sequitur.

- ▶ Customer conducts **release planning**, i.e., difficulty, cost & scheduling of realizing User stories.
- ▶ Real users are part of **whole team** for reqrs, priorities, and steering of proj.
- ▶ Customer defines **acceptance tests** for realized User stories; theses added into **regression** testing regime.
- ▶ Proj co-opts Customer's simple evocative vision of how new sw works, via use of **metaphor**.
- ▶ Customer sets reqrs **small** yet with **real business value**, so they can be realized and shipped to customer **frequently** as "shrink-wrapped" sw features.



Effective writing conventions: Three-C's

Prior to Agile conclave, XP co-creators never published their convention for writing User stories; thus first convention published was Connextra in May 2001. Only then did XP co-creator Ron Jeffries post Aug 2001 blog reaffirming stories original **non-format** convention, and gave it name, “**Card, Conversation, Confirmation**” or “**Three-C's**”.

- ▶ **Card.** Story is **unformatted first-hand outline**, by **authentic user**, who stays for all three-C's, of biz process with just enough info to evoke **one unique proto-reqr**. Story bound not by standard format, but by **standard-size** text box. (Original was 3"x5" **index card**.) If story too large to fit, then it must be revised to fit into **smaller** box; it encourages users to not write SRS-kind of reqrs. Story storage media can be **physical** or **digital**, as dictated by PB setup. Once validated, story in standard text box is used as **marker** in mapping its progress through realization.
- ▶ **Conversation.** This is **collaboration** of user and developer akin to **Eliciting** Method in RE RCM, to **specify full reqr** from 'proto' one and to **estimate realization** effort. It uses “social” eliciting techniques, exchanges **documents**, and demos **prototypes**. Prior to specification, **proto-reqr** is **validated**. **Priority** and **cost** notes are appended to validated User story, depending on available space. In parallel to specification, criteria for **acceptance** of sw realization is also generated.
- ▶ **Confirmation.** This is **realization** of sw by developer, followed by **acceptance testing** of said sw by authentic user akin to SDLC **User Acceptance Testing (UAT)** with two important differences: a) user already **familiar** with sw's capability and UX because pre-coding development work is done in full collaboration with developer, and b) only **one** reqr is being tested, to allow incremental release of accepted sw to customer.

Effective writing conventions: Connextra

In May 2001 during their new sw installation project for UK company Connextra, Rachel Davies and her XP development team conceived and applied new **format template** for effectively writing User stories. Named “**Connextra format**”, it is sentence comprising three parts that are filled in by stakeholder (see below; underlined & italicized words indicate type of entries by stakeholders).

As a position , I want to do biz process , so that benefit derived .

Tips for content style in each entry field:

- ▶ **Position.** Must be specific job appointment or title. Generic roles such as “user” or “manager” are insufficient to understand context and identify stakeholder (for conversation).
- ▶ **Biz process.** Work unit per breakdown of stakeholder’s job duties; not technical action such as pressing button. It must have measurable criteria of completion, although that is not shown with story itself.
- ▶ **Benefit.** Rationale or justification that is job-related, not personal. It can be any of: achieving biz goal, fulfilling biz purpose, or generating something of value to biz.

Acceptance tests may be appended to User story text box, i.e., index card, data element.

Effective writing conventions: INVEST

Independent consultant & XP early practitioner [Bill Wake](#) published third convention of User story effective writing in Aug 2003 [blog](#), “INVEST in Good Stories & SMART Tasks”. Acronym **INVEST** highlights characteristics that User stories of [good quality](#) will exhibit.

- ▶ **“I” ndependent.** Its content does [not overlap](#) with any others.
- ▶ **“N” egotiable.** Its exact representation of full reqr is completely open to being [elaborated via conversation](#). (Assume “conversation” meaning as per Three-C’s.)
- ▶ **“V” aluable.** Its derived benefit to authentic user must be [tangible & compelling](#).
- ▶ **“E” stimable.** It must have enough [prioritization](#) content to enable [cost & effort](#) derivation by orgs.
- ▶ **“S” mall.** It fits in given [standard text box](#) and its realization is achievable in [minimum](#) period of time.
- ▶ **“T” estable.** Its [operability](#) must be measurable, so definitive [acceptance](#) can be made of its [suitability](#).

User story authors must accompany them with [scorecard](#) showing which quality features are met and which aren’t. Poor scorecard doesn’t mean User story is discarded; more often, developers use it to [critique when and in what order](#) such User stories are processed.

Today's learning outcomes

One slide

Remember **why** knowing learning outcomes **important!**

Roadmap of learning goals & expectations.

Encourages developing preparation & study strategies.

Promote engaging in learning process.

Facilitate managing learning material & effort.

LO#11: UseCase2.0 reworked for Agile

After today's lesson, student will be able to **explain** ...

- ❑ ... UseCase2.0 methodology.
- ❑ ... **parent-UC** method including **Auxiliary details**.
- ❑ ... UseCase **Stories**.
- ❑ ... UseCase **Slices**.
- ❑ ... **effectively writing** Auxiliary details, Stories, and Slices.

After today's lesson, student will be able to **sketch artistically as opposed to formally** UseCase2.0 process model.

These are objectives I plan for you to achieve in this session.

Your objectives should sync with these, but don't need to be verbatim identical.

Today's **new** learning **material**

Twelve slides

History of UseCase original version

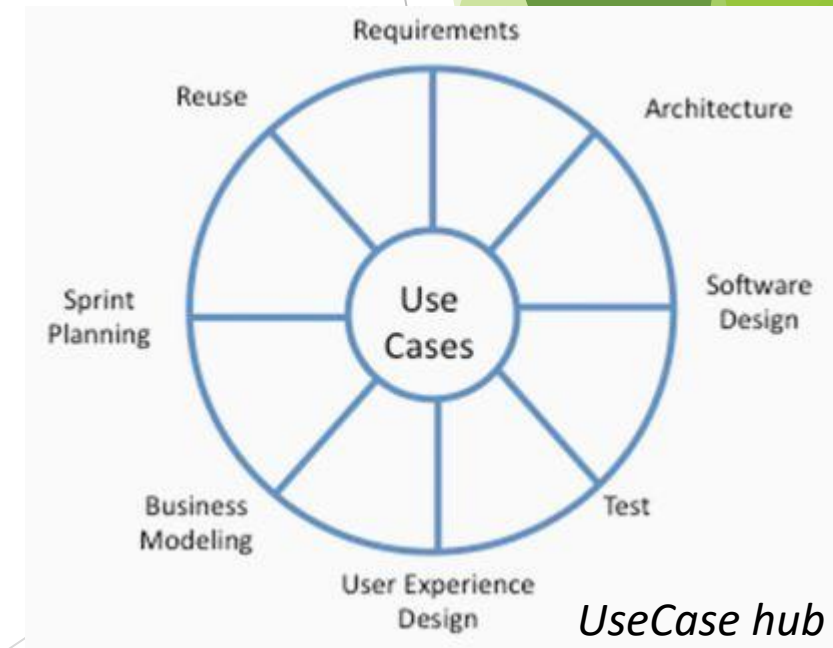
UseCase method is brain-child of Swedish computer scientist Ivar Jacobson, and has been in use for almost 60 years. From 1967, original modeling method coined “traffic cases” was used to specify functions and features for large telecommunication switching systems built and operated by Ericsson. Later, for application as foundational element in Objectory method for specifying sw reqrs, he enhanced traffic cases into what we now know as “UseCases”. In particular, said UseCases specified and modelled all ways of using system to yield particular goal, purpose, or value (GPR) for particular user.

At second annual OOPSLA in 1987, Ivar revealed concept of UseCases in his article “Object-oriented development in an industrial environment”, and how they were used at Ericsson to capture large system reqrs. In 1993, he along with Christerson, Jonsson & Overgaard authored book “Object-Oriented Software Engineering — A Use Case Driven Approach”. Among other major ideas, it introduced UseCases in graph & matrix representations as cornerstones of software design. After book’s publication, interest in applying UseCase method grew steadily.

History of UseCase original ver (cont)

After release of versions 0.9 and 0.91 of UML in 1996 — being first to include Jacobson's UseCases in diagram-only form — popularity of UseCase diagrams soon **eclipsed most of others** in specification. For instance, in 2006 article “How UML is used” by Dobing and Parsons, usage of UseCase diagrams was tied in second place with Sequence diagrams; Class diagrams were first. Interestingly, UseCase matrices, a.k.a., descriptions, were in third place, ahead of Activity, StateMachine, and Collaboration diagrams.

Unlike other reqrs capturing methods, UseCases can also be SDM, ergo, UseCase-driven 1) **SDM**, and 2) **SWE**. Its creator goes further: it can drive most of individual parts of **whole SDLC** (see right; even Agile SDM) as **framework**. E.g., Analysis stage utilizes UseCase **value statement** to generate Boundary-Control-Entity elements, or Architecture stage components are UseCases encapsulating business logic and interfaces, or Testing stage applies UseCase models to ascertain if newly realized sw is acceptable.



Anything you are uncertain about? ...
Ask, and let me help clarify

**Take
quick
break**

300 sec countdown – ends when fully filled

**Let's
continue
with LS**

Lightweight RCM applying combination of, **parent-UseCase** (UC) circa 1987-92,97, **Stories** similar to XP User stories, and **Slices** that constitute reqrs of new not yet existing sw; **subtype** of parent-UC. It comprises **six** steps, 1) **draw** UC **graphic** model and **list** '**Auxiliary details**' for big picture, 2) **prioritize** UCs, 3) **write** Stories covering complete range of ways each UC is used, 4) **bring forth** via conversation full **specification** of reqrs behind brevity of Stories, 5) **write** Slices to **align** select reqrs into "vertical" orientation set by Jacobson's **BCE** model, and 6) **prioritize** & deploy Slices for **realization**. Once realized, Slices are discarded, and new source code + UC model + Auxiliary details, become primary documentation for new sw.

UseCase2.0 methodology

Last of three Reqs Capture Methodologies that we will study our module.



Attention

UseCase2.0 is new application of UseCase

In 2011, Jacobson collaborated with Ian Spence and Kurt Bittner to write eBook, “Use-Case 2.0: The Guide to Succeeding with Use Cases”. It explained that UseCase2.0 is **new way** of applying UseCases that is lightweight, lean, scalable, versatile, easy-to-use, and **inspired by User stories**. Structurally, UseCase2.0 is **still UseCase 1987-92,97**, a.k.a., **parent-UC**, its **UML specification unchanged**, and retains its original strengths, especially its coverage across SDM, SWE, and SDLC. Now, with UseCase as **cornerstone**, they supplemented parent-UC with two new methods, **Story** and **Slice**.

Are “User story” & “Story” same? In 1996, Kent Beck (XP lead-creator) says he was **inspired** by Jacobson’s UseCases. In them, he saw that users elaborating **cases** about ‘value yielded by sw’, could establish with developers shared understanding on exactly what was needed. (Biggest challenge in sw development: communicating details of what to build.) So, name “User story” signifies how they’re used and not how they’re written. Jump 15 years later. With his UseCase2.0, Ivar says “**inspiration has flown in opposite direction**”.

In 2002, Kurt Bittner and Ian Spence wrote definitive book “Use-Case Modeling”. Ivar openly praised Spence as expert on Use-Cases. In 2011, all three co-wrote “Use-Case 2.0 Guide to Succeeding with Use Cases”. (It’s true that you see farther on shoulders of giants.)

UseCase: Cornerstone of this RCM

Factsheet of
fundamental
ideas about
User story
RCM

A specification of some part of biz ops of new sw that a) is stable, b) achieves observable GPV for either or both of user and new sw, and c) is performed in all sequences of related transactions akin to dialog with user(s). Typically, said biz ops is initiated by said user(s). Finally, it is weight-neutral RCM, being cornerstone of both heavy and light.

Stable. It's unreal that any biz ops yielding value to both user & system (the “what”) would change in parallel with development of new sw (the “how”). Only exception is ‘Business Process Reengineering’.

GPV. (Acronym for “goal, purpose, value” in LS#4) This drives business to strive in its efforts to efficiently produce products or services that customers consistently find useful, effectively align with market demands, and steadfastly achieve financial success.

Dialog. Transactions between user and system are not portrayed as one-sided, like in maps or code. Instead, they are articulated as dialog, e.g., user does, system responds, user counters, and so on.

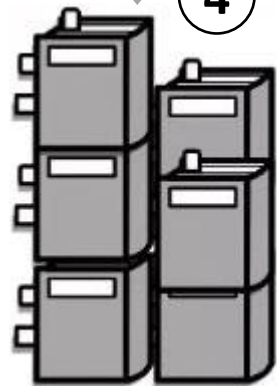
Weight-neutral. UseCase has two models, graphic and matrix. In heavyweight project, both models are valuable additions to project documentation. In lightweight, graphic model only (no matrix) for providing big picture for Stories & Slices. Once all are generated, discarding or keeping them is moot.

Users **prioritize** each UC for MVP (2)

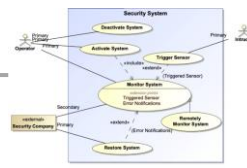
Users write **Stories** for **priority** UC (3)

Story

Face to Face communication tabli p4,
Team's Self organization & Accourty
Transparency, Openness in stands
reviews, and retrospectives p5, p10
Adaptability to change find find i,
Fixing problems without waiting t
broke it or Who should fix it



Store Stories in **PB**
per UC & priority



(1) Users draw **Use-Case model** of new sw, and list **Auxiliary details**



Users



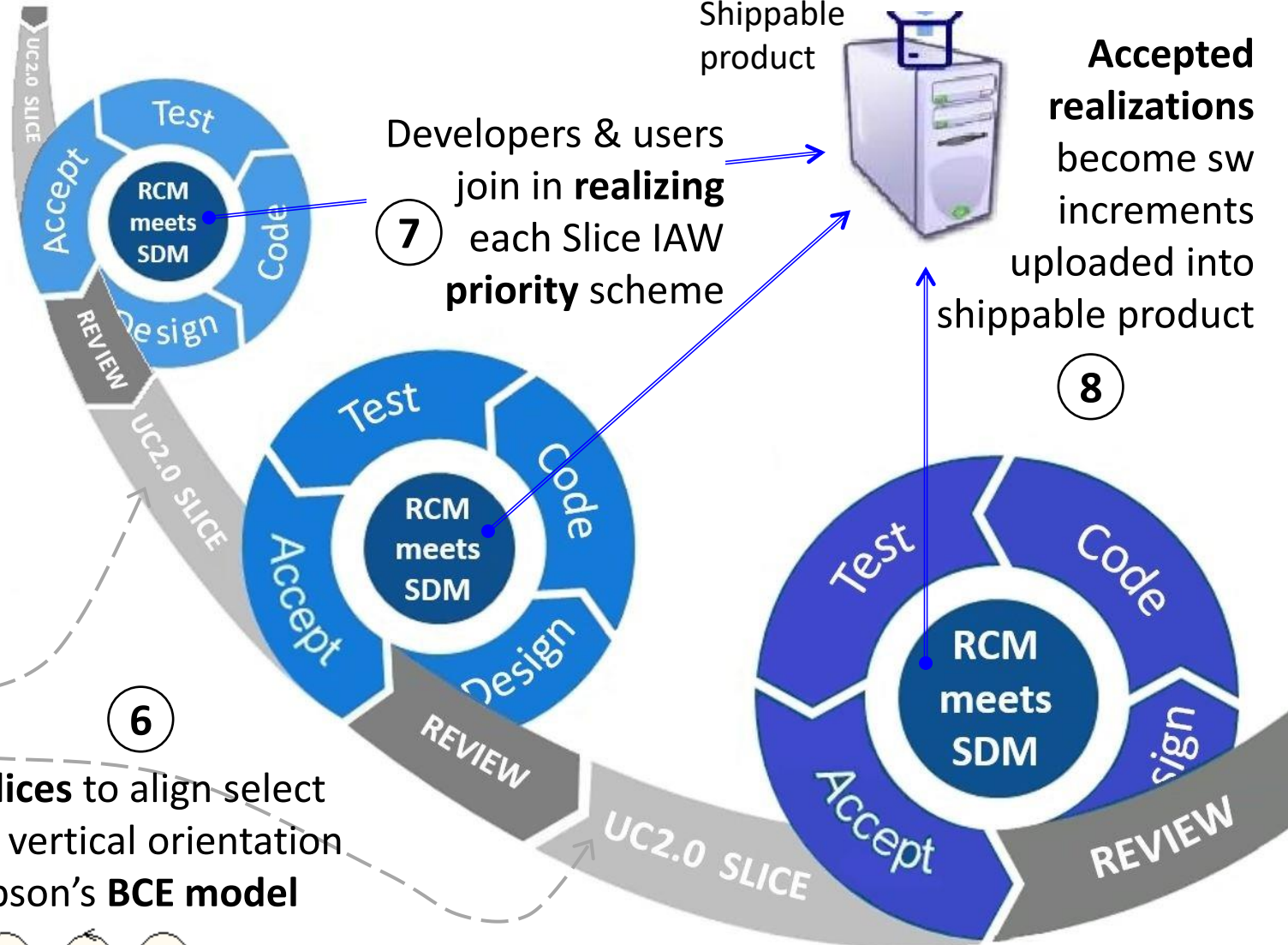
Developers

Developers join users to **specify** full reqrs from Stories

(5)

Slices

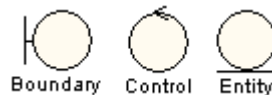
UseCase2.0 process model



Developers & users join in **realizing** each Slice IAW **priority** scheme

Accepted realizations become sw increments uploaded into shippable product

Write **Slices** to align select reqrs in vertical orientation of Jacobson's **BCE model**



Auxiliary details: what & how to write them effectively

UC2.0 processing model slide shows that UC graphical model is accompanied by [Auxiliary details](#) list. It's also stated earlier that matrix form of UC model is not used in lightweight development. So, how to retain still-critical info contained therein, yet conform to Agile principle of “working sw over documentation”?

Write said critical info — assumptions, business rules, constraints, pre- & post-conditions, and technical details — into [simple list](#) with same rules as ‘[note](#)’!

Creator of UC2.0 provided only guidelines about having critical info available. For academic purposes, [makeshift template](#) is appropriate (see right for example.) Content for each field is [free-text](#); if nil, “none”. Add or delete technical items as applicable.

Auxiliary details

For: < <“Model of new sw” <name>> or
< “UseCase” <name>> or
<“Story” <name> > >.

Effective date: in Julian format as per policy.

List of details:

1. **Assumptions:**
2. **Business rules:**
3. **Constraints:**
4. **Pre- & post-conditions:**
5. **Technical details:**
 - a. **Database elements & parameters:**
 - b. **Load & stress parameters:**
 - c. **Performance benchmarks:**
 - d. **Platform & IT compatibility:**
 - e. **Quality features:**
 - f. **UI & UX:**

Story & Slice: Keepers of RCM's fluidity

Story. Short description of dialogical transactions between user and new system; in other words, a 'story about way of usage' of UC. It is authored by user or BA. UC can have several Stories covering all ways that usage occurs in it; crucially, all must semantically relate to GPV of their UC. There are three categories of usage, basic, alternate & exception. (Definitions are per parent-UC specification, as is one only is basic.) Finally, while UC has pre- and post-conditions, Story may not; even if it did, neither would be expressed by author in those terms.

Slice. Technical specification of small portion of UC's functionality that undergoes realization per lightweight norms. Said functionality is drawn from UC's Stories, and established into an end-to-end vertically-oriented segment. Segment's orientation is derived from Boundary, Control, Entity (BCE) elements comprising Jacobson's Analysis Model (1992), and content from platform's sw architecture.

Stories are analysed for their BCE elements to determine if 1) whole Story connects through whole vertical stack, and if there are none, then 2) isolate parts of Stories to assemble vertical stack that fills out particular function. Ergo, it is usual to find that some Stories may be shared in several Slices. Criterion underpinning said analysis is that Slice should not be so comprehensive or complicated that long period of time is required to realize it.

Prioritization scheme for UC and Slice

UC2.0 processing model slide shows that UC from **graphic** model and **Slices** are subjected to **prioritization**. Said prioritization conducted per one of **three** schemes: minimum marketable product (**MMP**), minimum viable product (**MVP**), and **Walking skeleton**. Their common purpose is to **minimizes risk** and **validate** reqrs & assumptions **early** to avoid painful rewrites later, but with specific emphases:

- ▶ **MMP**. Coined by Mark Denne & Jane Cleland-Huang in 2003 “Software by Numbers: Low-Risk, High-Return Development”. For businesses that outsource development of product that they intend to sell or distribute. Must be stable and polished enough to provide **reliable test market** feedback while protecting **brand trust**.
- ▶ **MVP**. Coined by Frank Robinson at SyncDev in 2001, and later popularized in 2011 “The Lean Startup”. Has enough features for early adopters to **meaningfully use** and provide **feedback** on what may need adjustment.
- ▶ **Walking skeleton**. Coined by Alistair Cockburn in 2004 “Crystal Clear: A Human-Powered Methodology for Small Teams”. Technical **proof of concept** of architecture with minimal features; **backbone** of development.

Generally, Walking skeleton is realized **first** before either of MVP or MMP, ensuring that decisions pertaining to biz ops can at least have foundation of new product’s technical viability. Prioritization is **not** static; it can be redirected as more increments of shippable product are released.

Alistair's effective writing of Stories

[Alistair Cockburn](#), independent consultant, and co-signatory on 2001 Agile Manifesto, started his decades-long association with writing UseCases when he visited C3 project in 1998, expecting Use cases (see LS#10). In 2000/2001, he published his tome “Writing Effective Use Cases”, cementing his eminence in writing UseCases. (Interestingly similar to way Mike Cohn gained his estimable status in effective writing of User Stories.)

UseCase [templates](#) in his 2000/2001 book are elaborations of varying complexity of [matrix models](#). As such, they are not appropriate for lightweight development, except for one of them, i.e., ‘[Casual](#)’ & ‘[low-ceremony](#)’ (see below). Essentially, this is most useful to UC2.0 for effective writing of [Stories](#).

Use Case 1: Buy something

The Requestor initiates a request and sends it to her or his Approver. The Approver checks that there is money in the budget, check the price of the goods, completes the request for submission, and sends it to the Buyer. The Buyer checks the contents of storage, finding best vendor for goods. Authorizer: validate approver's signature. Buyer: complete request for ordering, initiate PO with Vendor. Vendor: deliver goods to Receiving, get receipt for delivery (out of scope of system under design). Receiver: register delivery, send goods to Requestor. Requestor: mark request delivered.

At any time prior to receiving goods, Requestor can change or cancel the request. Canceling it removes it from any active processing. (delete from system?) Reducing the price leaves it intact in process. Raising the price sends it back to Approver.

Effective writing of Slice

Being technical specification involving database elements, Interface contracts & platform's sw architecture, Slice is written more **formally** and with more **technical jargon** than Stories. But, as with Stories, Slices have no accepted writing convention. Instead, it has only guidelines from its creator.

For academic purposes, **makeshift convention** is appropriate. (See right) Said guidelines by Jacobson are converted into **Slice Technical Specification (STS) template**. Examples provided in next slide.

Title: Slice unique ID and name.

Context:

- **Effective date:** Date in Julian format as per policy.
- **Parent UC:** ID and name of parent UC.
- **Story list:** Prioritized **list** of the specific Stories used.
- **Prelim:** List of pre-conditions plus description of current situation to ready for realization according to its dictates.

Narrative: Derived telling of Stories sourced for vertical stack. Also include Auxiliary details specific to used Stories.

Scope:

- **Category:** basic / alternate / exception.
- **Priority, MVP, Status, KIV, , etc.**

Tech details: Details of actions, including changes, to specific Architectural components in vertical stack, including UI (Boundary), Logic (Control), and Database (Entity). Also API/Interface contracts between Layers, and from internal system to external. Also, infrastructure setup, DevOps and environmental requirements necessary to deploy and run Slice.

Acceptance: List of Acceptance criteria (AC) tests, including binary checks {Y, N}, to confirm the Slice's vertical stack is 'done/fully in place'.

Slice effective writing examples

Walking skeleton

Title: SYS-SLICE-01: Global Stack Verification.

Context:

Effective date: 16 Nov 2021.

Parent UC: UC-01 System readiness and confirmation.

Story list: S01 & S02.

Prelim: The developers have isolated parts of the 'connection Stories' to build the first Slice.

Narrative: The user triggers system heartbeat from the dashboard; the system passes this through the API to DB and returns timestamp, confirming the full vertical path is live.

Scope:

Category: Basic.

Status: MVP.

Priority: blocker [Essential for all subsequent work].

Technical:

UI: "System Status" widget with a 'Test' button.

Logic: PingController calls HealthService.

Data: Write/Read test to System_Status table(in Postgres).

Acceptance:

AC01: Dashboard displays "Stack Online" after button click.

AC02: Log entry created in DB with correct Server_ID.

Exception, still part of MVP

Title: INT-SLICE-09: Postal API Timeout Handler.

Context:

Effective date: 21 Nov 2021.

Parent UC: UC-05 External System Utilization & Management.

Story list: S05 & S06.

Prelim: This slice is vertical stack assembled specifically to handle failure state when external system times out.

Narrative: When external validation service fails to respond, system catches timeout at logic tier and presents "Manual Verification Required" override to the user at GUI tier.

Scope:

Category: Exception.

Status: MVP.

Priority: High (Prevents blocked sales).

Technical:

UI: Modal popup for "Manual Override" signature.

Logic: ExternalServiceWrapper with 5-second CircuitBreaker and exception catch.

Data: Set Validation_Method to "Manual_Override" in Orders table (in Postgres).

Acceptance:

AC12: If API is blocked (simulated), "Manual Override" modal appears within 6 secs..

AC14: Transaction proceeds to "Authorized" status despite external failure.

Active learning

Three slides

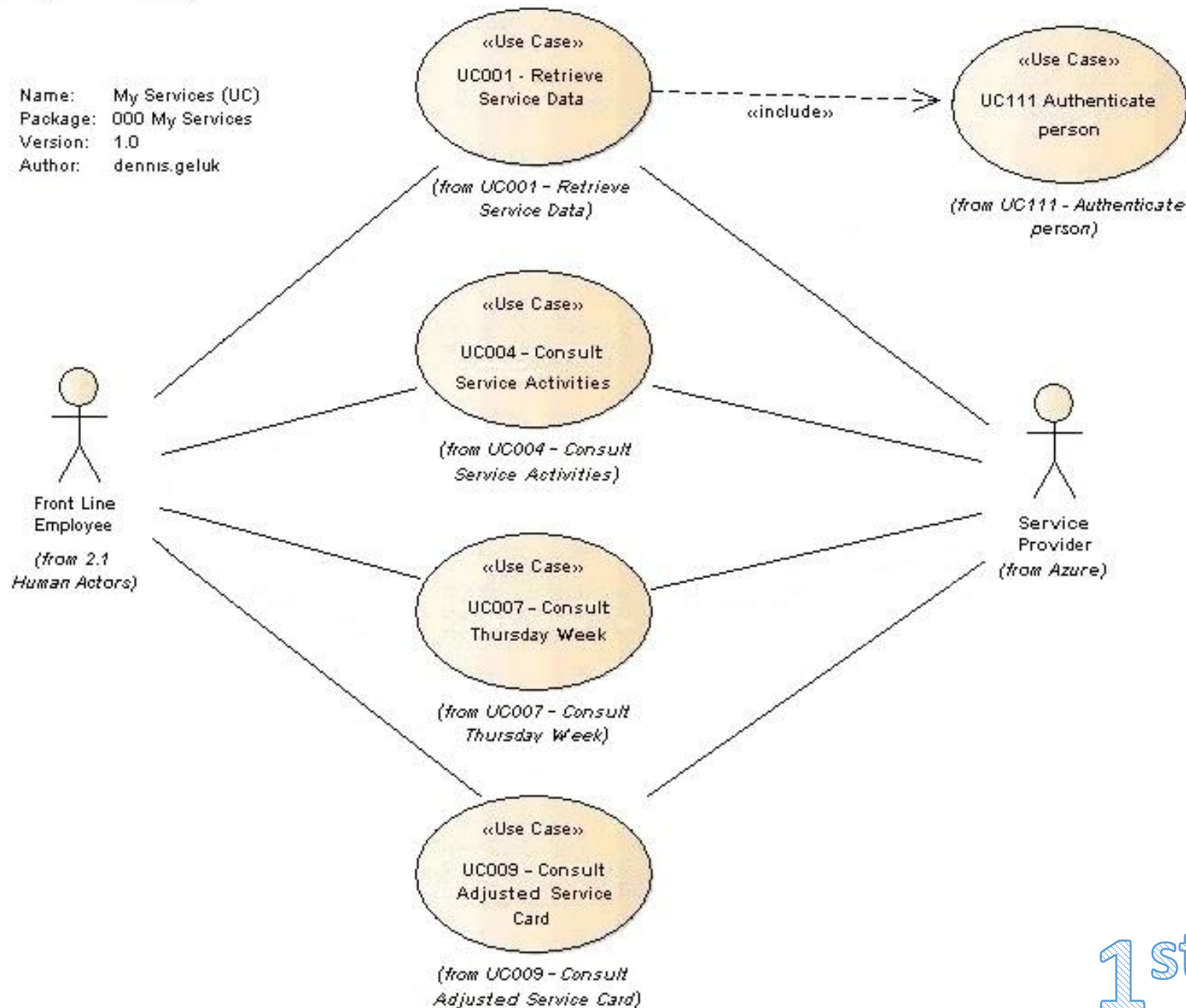
RailPocket: From PDA to Smart phone app

Dutch Railways is principal passenger railway operator in Netherlands. To facilitate real-time improvement of its passenger services and experience, in February 2007 it fielded fleet of **PDAs** named **RailPocket** for use by its 10,000 front-line employees and 50 service ambassadors to do ...

- ▶ **Ticket validation.** Conductors can read and validate passenger tickets in minutes.
- ▶ **Providing updates.** Employees can inform travelers in near real time with latest info on its trains and stations, such as schedules, onboard quantities, queuing times, and on-route station bypasses.
- ▶ **Internal data entry.** Conductors and engineers can immediately report problems and get action taken on problems that may take hours or days, such as seating capacity, train punctuality, and safety or maintenance issues.

Though highly successful (and award-winning), it was bulky to carry, it was slow, its operating system was no longer supported, and it was operating at its capacity limit leaving no possibility of expanding its functionality. Dutch Railways launched project in 2014 to replace PDA with **series of Railway apps** powered by ubiquitous Smart phone. To conduct development using **Agile SDM**, project adopted **UseCase2.0** for its RCM. Project was wildly successful!

Name: My Services (UC)
 Package: 000 My Services
 Version: 1.0
 Author: dennis.geluk

1st

Instructions

Given RailPocket replacement project is gold standard of applying UC2.0, our company wants to replicate that winning formula in our own project pipeline. Your team will be giving company's workshops on UC2.0. In preparation, you searched for Rail-Pocket proj **models** but found only:

- ▶ 1st model is "MyShift" **app**.
- ▶ 2nd model is big picture of **Slices** for UC001 Retrieve Service Data.

Your RO wants ProgRep. Your team decides to make **short slide prez** of all models in UC2.0 process, mixing **existing** with **templates of missing models** (that you will fill in later), in **correct order** as per process. Post

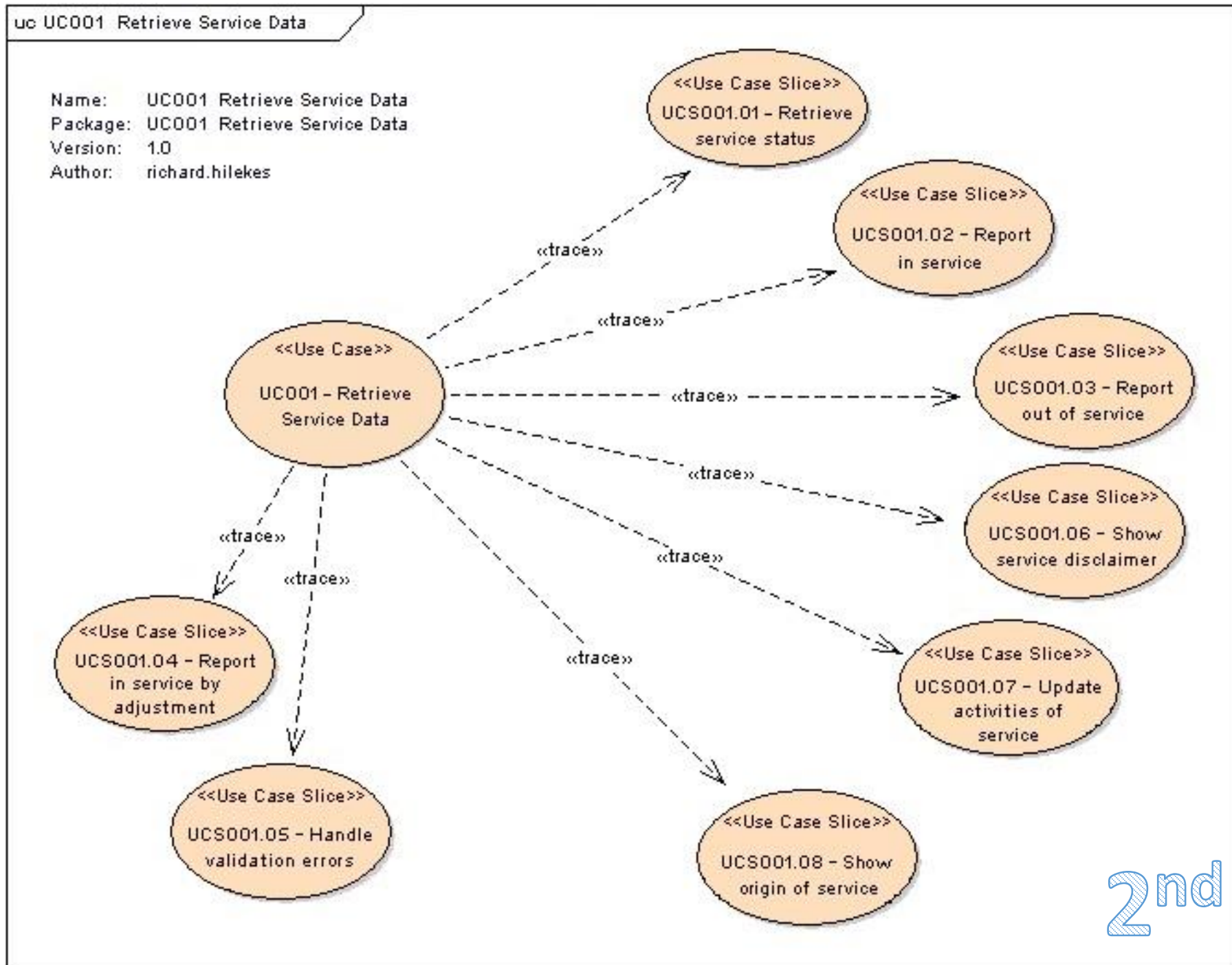
Instructions (cont)

You have 15 mins to prepare.
Just sketch outline of models
& SnipIt into slides.

Your team has less than two
mins to present. Identify
each model and what they
represent in UC2.0 RCM.
You'll be critiqued on
explanation and order of
models.

Assume your prioritization
scheme is walking skeleton.

At least three teams will be
selected in quick succession.
Feedback is available from
teacher (being RO) & peers.



Epilogue

Two slides

UseCase2.0 emerges as **phenomenal alternative** to User stories, by bridging gap between high-level system intent vs granular, iterative delivery, through its unique three-method approach. First, it **preserves reliable parent-UC**, providing stable architectural anchor that ensures “big picture” remains visible entire project duration. Second, it effectively re-works “usage stories” by making them into **prioritizable dynamic flows** that anticipate complexity before it hits realization effort. Third and most transformative is exceptional method of “slicing”, **vertically-oriented Slices of functionality** that provide direct pipeline from value to specification to realization, ensuring that every iteration delivers functional piece of new sw that is both architecturally sound & immediately valuable to user.

“Big things have small beginnings.”

My final words

Contact me for ...

- ▶ regular stuff via my work email h51581@singaporetech.edu.sg.
- ▶ more sensitive stuff, by Whatsapp [97761716](tel:97761716) or my personal email johnmilton2000@gmail.com.

Tomorrow's lab is 2nd on subject of [Validating](#). Hopefully, teams are comfortable with all data they've collected about errors in the SRS, and those in vendor subprocesses. In preparation, ensure you are familiar with your ...

- ▶ [Validating report](#) details to date.
- ▶ presentation as [QC](#) team member to [Specifying team/developers](#) who wrote SRS.
- ▶ presentation as [QA](#) team member to [vendor's C-staff](#) in charge of [RE](#) conduct.

Keep on working through [Glossary](#), four terms per week, and Google.

Teachers take certain concepts being understood for [granted](#). If you are falling behind, see me or TA for [consultation](#).

Glossary & Google readings for next week

□ ???

Any final thoughts?

If not ...

End of LS#11

UseCase2.0: Heavyweight RCM Updated To Lightweight
by Prof Kevin

ICT2113 Software Reqs: Now & Near Future